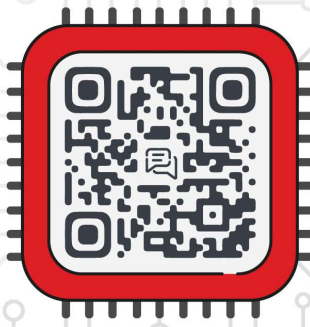




14

10

2100 sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة

الاستقراء البنيوي
والتقطيع البنيوي



د. عمار جوخدار

محتوى مجاني غير مخصص للبيع التجاري

RB^s Informatics; 01/06/2026

برمجة تفرعية

مراجعة و تذكرة بالمحاضرات السابقة

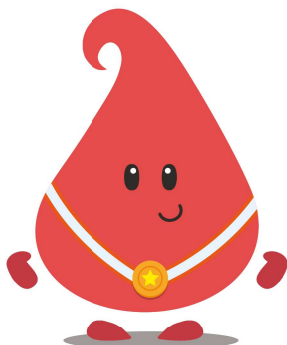
تذكرة بالمحاضرة الثامنة:

تأثير الفهارس على الأقفال :

تنسب الفهارس (Indexes) في زيادة الأقفال (Locks). فالحفاظ على اتساق البيانات عند تعديل سجل معين لا يضع قفلا على السجل فحسب، بل يمتد القفل ليشمل جميع السجلات الأخرى التي تشاركه نفس قيمة الفهرس (Index Value).

امتدادا لهذا السياق:

- في المحاضرة السابقة تابعنا دراسة المشكلات المرتبطة بالأنظمة المتوازية وقواعد البيانات، وركزنا على مفهوم **إدارة التنافس (Concurrency Management)** باعتباره أحد العناصر الأساسية في بناء الأنظمة الحديثة. وقد رأينا أن منع حدوث التنافس والأخطاء الناتجة عنه أمر ضروري للحفاظ على صحة البيانات وسلامة النظام، إلا أن تحقيق ذلك يجب ألا يكون على حساب أداء التطبيق أو قدرته على العمل والاستجابة للمستخدمين.
- كما تناولنا بعض القضايا المتعلقة بإدارة قواعد البيانات الموزعة، وآليات معالجة التنافس عند وجود أكثر من نسخة من البيانات، إضافة إلى المشكلات التي قد تنتج عن التزامن بين الأنظمة المختلفة.
- بعد ذلك انتقلنا إلى دراسة استهلاك الموارد في الأنظمة البرمجية، وتعرفنا على مجموعة من أنماط التصميم التي تهدف إلى تحسين الأداء وتقليل استهلاك الذاكرة والمعالج والشبكة. ومن أهم هذه الأنماط **Lazy Design Patterns** التي تعتمد على تنفيذ العمليات أو جلب البيانات عند الحاجة الفعلية إليها فقط.



ما تفعله عندما لا يراك أحد... هو ما يصنعك حقاً.

ومن الأمثلة التي تمت دراستها:

1. Lazy List:

التي تعتمد على جلب البيانات تدريجياً بدلاً من تحميلها كاملة دفعة واحدة.

2. Frame Pattern:

التي تمثل أحد أشكال الـ Lazy Design، حيث تعتمد على التعامل الذكي مع البيانات من خلال ثلاث آليات أساسية (العقاريت الثلاثة):

- If Needed
- If Added
- If Removed

وقد ظهرت فكرة الـ Frame أساساً في مجال الذكاء الاصطناعي بهدف محاكاة طريقة تفكير الإنسان، فالإنسان لا يحاول تخزين أو معالجة جميع المعلومات دفعة واحدة، وإنما يتعامل مع المعلومات المطلوبة فقط عند الحاجة إليها. ومن هنا جاءت فكرة الحقول الديناميكية والحالة الثالثة للقيم (Unknown) التي تميز بين المعلومة غير الموجودة والمعلومة غير المعروفة.

كما ناقشنا العلاقة بين أنماط التصميم وموثوقية البرمجيات (Reliability)، وتوصلنا إلى أن أحد أهم عوامل زيادة موثوقية البرامج هو **تقليل حجم الكود البرمجي**. فكلما كان الكود أصغر وأسهل في الفهم والمراجعة، أصبحت عملية اكتشاف الأخطاء وتصحيحها أكثر سهولة. ولهذا السبب تعتمد الأنظمة الحديثة على مفاهيم مثل:

- Low-Code Frameworks
- Design Patterns
- Framework Engines

وذلك بهدف تقليل كمية الكود المكتوب يدوياً وزيادة الاعتمادية.

وفي نهاية المحاضرة تناولنا تصنيف البرمجيات اعتماداً على طبيعة المدخلات التي تتعامل معها، حيث يمكن تقسيمها إلى ثلاثة مستويات رئيسية:

1. الأنظمة القابلة للعد (Countable Systems):

وهي الأنظمة التي يمكن وصف جميع حالاتها باستخدام قواعد وشروط محددة.

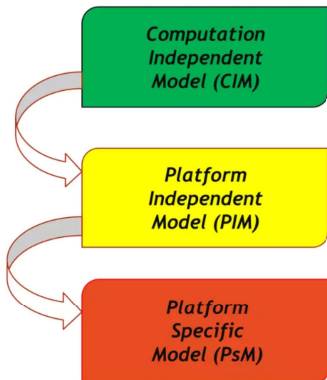
2. الأنظمة القابلة للعد بشكل عودي (Recursively Enumerable Systems):

والتي تشمل معظم البرامج التقليدية مثل المترجمات (Compilers)، حيث يمكن وصف عدد غير منتهٍ من الحالات باستخدام مجموعة محدودة من القواعد الرسمية.

3. الأنظمة غير القابلة للعد (Non-Enumerable Systems):

وهي الأنظمة التي تتعامل مع الصور والأصوات والفيديو واللغة الطبيعية، وتشكل الأساس الذي تقوم عليه تطبيقات الذكاء الاصطناعي وتعلم الآلة.

البنية الموجهة بالنماذج (Model-Driven Architecture - MDA)



يهدف MDA إلى تقليل كمية الكود التي يكتبها المبرمج يدوياً، وزيادة الاعتماد على النماذج (Models) والتحويل الآلي بينها، مما يؤدي إلى أنظمة أكثر موثوقية وأسهل في الصيانة والتطوير. كما يعد هذا الأسلوب مناسباً بشكل خاص لمعالجة الأنظمة التي تنتمي إلى فئة الأنظمة القابلة للعد بشكل عودي (Recursively Enumerable Systems)، والتي تشمل معظم التطبيقات البرمجية التقليدية.

تعتمد MDA على تقسيم عملية تطوير النظام إلى ثلاث مراحل أو مستويات رئيسية، يبدأ كل منها بدرجة مختلفة من التجريد.

أولاً: النموذج المستقل عن الحساب (Computation Independent Model - CIM)

يمثل الـ CIM مرحلة التحليل وفهم منطق العمل (Business Logic)، وهو النموذج الذي يصف ما يجب أن يفعله النظام دون أي اهتمام بكيفية تنفيذه برمجياً. في هذه المرحلة يتم التركيز على فهم الواقع وقواعد العمل الحقيقية، ويستخدم عادةً وصف بلغة طبيعية مفهومة من قبل الخبراء والمختصين في المجال. فعلى سبيل المثال، عند تطوير نظام جامعي أو نظام لمشفى، يتم في هذه المرحلة وصف القواعد والإجراءات المتبعة في الواقع، مثل:

- شروط نجاح الطالب أو ترفيعه.
- آلية تسجيل المقررات.
- قواعد القبول والتخرج.
- الإجراءات الإدارية المختلفة.

ويتميز هذا المستوى بأنه مستقل تماماً عن لغات البرمجة وقواعد البيانات والمنصات التقنية.

أهمية الـ CIM:

يُعد الـ CIM أهم مرحلة في عملية التطوير، لأن أي خطأ في فهم الواقع أو تحليل متطلبات العمل سينتقل إلى جميع المراحل اللاحقة.

فإذا كانت القواعد الموضوعية في هذه المرحلة ناقصة أو غير صحيحة، فإن النظام النهائي سيكون خاطئاً مهما كانت جودة البرمجة أو مستوى الذكاء الاصطناعي المستخدم في توليده.

مثال توضيحي:

لنفترض أن شخصاً أراد تصميم نظام لإدارة شؤون الطلاب في كلية المعلوماتية، فكتب قاعدة بسيطة تقول:

- "يُرفع الطالب إذا كان عدد المواد التي رسب بها أقل من أربع مواد."

قد تبدو هذه القاعدة صحيحة ظاهرياً، لكنها في الحقيقة قد تكون ناقصة لأنها أهملت العديد من التفاصيل المهمة، مثل:

- المواد الإدارية.
- قوانين المساعدة الامتحانية.
- الترفيع الفصلي وشروطه.
- العلاقات بين المقررات والمتطلبات السابقة.
- الحالات الاستثنائية التي ينص عليها النظام الجامعي.

في هذه الحالة، حتى لو استخدم المطور أحدث أدوات البرمجة أو طلب من نظام ذكاء اصطناعي توليد التطبيق كاملاً، فإن النتيجة ستكون نظاماً خاطئاً؛ لأن المشكلة الأساسية موجودة في مرحلة التحليل نفسها.

- لذلك فإن نجاح أي مشروع برمجي يبدأ أولاً من بناء CIM صحيح وكامل.

ثانياً: النموذج المستقل عن المنصة (Platform Independent Model - PIM)

بعد التأكد من صحة منطق العمل، يتم الانتقال إلى مرحلة تحويل الوصف الطبيعي إلى نموذج صوري (Formal Model).

في هذه المرحلة يتم التعبير عن القواعد بطريقة دقيقة وقابلة للمعالجة الحاسوبية، باستخدام:

- الرموز.
- العلاقات المنطقية.
- القواعد الرياضية.
- النماذج الشكلية.

فعلى سبيل المثال، بدلاً من كتابة:

- "ينجح الطالب إذا حصل على علامة جيدة".

يتم تحويلها إلى قاعدة صريحة مثل:

- Note >= 60

وهنا يبدأ دور مهندس البرمجيات الذي يجب أن يكون قادراً على فهم اللغة الطبيعية وتحويلها إلى قواعد صورية دقيقة يمكن للحاسوب التعامل معها.

أي تحويل الـ CIM إلى PIM.

ويتميز الـ PIM بأنه ما يزال مستقلاً عن لغة البرمجة أو نظام التشغيل أو قاعدة البيانات المستخدمة.

مستقبلك يُبنى بما تفعله اليوم، لا بما تنوي فعله غداً.

ثالثاً: النموذج الخاص بالمنصة (Platform Specific Model - PSM)

بعد بناء النموذج الصوري الصحيح، يمكن تحويله إلى نموذج خاص بمنصة تنفيذ محددة. في هذه المرحلة يتم ربط القواعد الموجودة في الـ PIM بالتفاصيل التقنية الخاصة بالمنصة المستهدفة، مثل:

- Java
- C++
- JavaScript
- NET.

أو أي بيئة تنفيذ أخرى.

ويحتوي الـ PSM على جميع التفاصيل اللازمة لتنفيذ النظام على منصة معينة، مع المحافظة على منطق العمل الذي تم تعريفه في المراحل السابقة.

التحويل الآلي بين المراحل

من أهم مزايا MDA أن الجزء الأصعب من العمل يتم في مرحلتين:

- CIM
- PIM

أما بعد الوصول إلى نموذج صوري صحيح، فيمكن استخدام محركات (Engines) وأدوات متخصصة لتوليد أجزاء كبيرة من الكود البرمجي بشكل آلي. وبذلك يتم تقليل كمية الكود المكتوب يدوياً، وتقليل احتمالية الأخطاء، ورفع موثوقية النظام.

فوائد MDA:

تُعد MDA من أهم الأساليب الحديثة في تطوير البرمجيات لأنها تركز على رفع موثوقية النظام وتقليل حجم الكود البرمجي من خلال الاعتماد على النماذج بدلاً من البرمجة التفصيلية المباشرة. تبدأ هذه الفائدة من مرحلة النموذج المستقل عن الحساب (CIM) حيث:

1. يُبذل جهد كبير لفهم منطق العمل (Business Logic) وتحليله بلغة طبيعية قبل البدء بأي تفاصيل تقنية، مما يزيد الثقة بصحة المتطلبات و يقلل احتمال بناء نظام خاطئ بسبب سوء فهم الواقع.
2. بعد ذلك يتم تحويل هذه القواعد إلى نماذج صورية يمكن معالجتها آلياً، بحيث يكتب المهندس بتعريف القواعد والعلاقات الأساسية، بينما تتولى المحركات البرمجية توليد أجزاء كبيرة من الكود لمنصات مختلفة.

كما تسمح MDA بمعالجة عدد كبير جداً من الحالات الواقعية انطلاقاً من مجموعة محدودة من القواعد، بدلاً من كتابة أعداد هائلة من تعليمات If-Then-Else.

ومن أهم مزاياها أيضاً اعتماد مفهوم النموذج الغني (Rich Model):

الذي يجعل النموذج المصدر الوحيد للحقيقة، حيث تُعرّف القيود والقواعد مرة واحدة داخله، ثم تُستخدم تلقائياً لتوليد واجهات المستخدم المناسبة، وقيود قواعد البيانات، وآليات التحقق من صحة البيانات والتنبيهات البرمجية.

- ونتيجة لذلك تصبح النماذج أسهل في القراءة والفهم من الكود التقليدي المليء بالشروط المتداخلة، مما يسهل الصيانة والتطوير ويقلل احتمال ظهور الأخطاء.
- وبفضل أتمتة العديد من المهام الروتينية، يستطيع المهندس التركيز على التصميم والتحليل بدلاً من كتابة التفاصيل التنفيذية المتكررة، الأمر الذي يؤدي إلى تقليل زمن التطوير، ورفع الإنتاجية، وزيادة موثوقية البرمجيات.

MDA وطرق التفكير المنطقي الصوري

لا تعتمد MDA على توليد الكود فقط، وإنما تستند أيضاً إلى مفاهيم من المنطق الصوري (Formal Logic) الذي يهدف إلى تمثيل المعرفة والقواعد بطريقة تسمح للحاسوب بمعالجتها بشكل منهجي. وتظهر أهمية هذا الأمر لأن معظم الأنظمة البرمجية تتعامل مع مجموعات قابلة للعد بشكل عودي (Recursively Enumerable Systems)، أي أن عدد الحالات التي قد تواجهها غير محدود، لكن يمكن وصفها بعدد محدود من القواعد.

ومن أهم أساليب التفكير المنطقي المستخدمة في هذا المجال:

1. الاستنباط (Deduction):

يعتمد الاستنباط على تطبيق قواعد معروفة للوصول إلى نتائج مؤكدة.

فإذا كانت لدينا القاعدة:

إذا تحقق A فإن B يتحقق.

وكانت A صحيحة بالفعل، يمكن الاستنتاج مباشرة أن B صحيحة أيضاً.

ويعرف هذا الشكل المنطقي باسم Modus Ponens، ويستخدم بكثرة في الأنظمة الخبيرة (Knowledge-Based Systems) وفي تطبيقات اتخاذ القرار المعتمدة على القواعد.

كما يوجد شكل آخر يسمى Modus Tollens يعتمد على نفي النتيجة للوصول إلى نفي السبب.

$\frac{p \rightarrow q \quad p}{\therefore q}$ Modus Ponens	$\frac{\sim q \quad p \rightarrow q}{\therefore \sim p}$ Modus Tollens
--	---

2. (Abduction):

يعتمد Abduction على الوصول إلى تفسير محتمل للظاهرة، وليس إلى نتيجة مؤكدة.

فعلى سبيل المثال، إذا كانت الحمى أحد أعراض مرض معين، فإن وجود الحمى لا يثبت الإصابة بهذا المرض بشكل قطعي، وإنما يجعلها احتمالاً مرجحاً.

لذلك يستخدم هذا النوع من الاستدلال في:

- الأنظمة الخبيرة.

- المنطق الضبابي (Fuzzy Logic).

- الأنظمة الطبية والتشخيصية.

حيث تكون النتائج احتمالية وليست يقينية.

3. الاستقراء (Induction):

يعتمد الاستقراء على الانتقال من الملاحظات الخاصة إلى قاعدة عامة.

فعندما نلاحظ حدوث ظاهرة معينة بشكل متكرر، نقوم ببناء قاعدة عامة تتوقع استمرار حدوثها مستقبلاً.

ومن الأمثلة البسيطة على ذلك ملاحظة شروق الشمس يومياً، ومن ثم استنتاج قاعدة عامة بأنها ستشرق في اليوم التالي أيضاً.

ويشكل الاستقراء الأساس النظري للعديد من تطبيقات تعلم الآلة (Machine Learning)، حيث يتم بناء النماذج انطلاقاً من البيانات والملاحظات السابقة.

العلاقة مع MDA:

تكمن أهمية هذه المفاهيم في أن الأنظمة البرمجية لا تتعامل مع عدد محدود من الحالات، بل مع عدد كبير جداً قد يصل إلى اللانهاية عملياً. ولذلك لا يتم تمثيل كل حالة بشكل منفصل داخل البرنامج، وإنما يتم تعريف مجموعة من القواعد والنماذج القادرة على توليد هذه الحالات ومعالجتها.

لذلك يركز منهج MDA على بناء النماذج (Models)، لأنها تمثل النظام بمستوى تجريدي أعلى و أسهل للفهم و التعديل من الكود البرمجي، حيث تصف الكيانات و العلاقات و قواعد العمل، ثم يمكن استخدام هذه النماذج لتوليد أجزاء كبيرة من التطبيق بصورة آلية.

النموذج الغني (Rich Model)

يرى منهج MDA أن الجزء الأهم في النظام ليس كمية الكود المكتوبة، وإنما كمية المعرفة الممثلة داخل النموذج نفسه. ولذلك يركز على بناء ما يسمى **النموذج الغني (Rich Model)**.

يقصد بالنموذج الغني أن يحتوي النموذج على أكبر قدر ممكن من المعلومات والقواعد والقيود المتعلقة بالبيانات، بدلاً من توزيع هذه المعلومات بين الواجهات والتوابيع وقواعد البيانات.

فعلى سبيل المثال، بدلاً من تعريف حقل "الاسم" فقط داخل الكلاس، يمكن تعريف معلومات إضافية مثل:

- أن القيمة إلزامية.

- أن المحتوى يجب أن يكون أحرفاً فقط.

- أن القيمة يجب أن تكون فريدة.

- أن طول النص يقع ضمن حدود معينة.

كما يمكن تعريف قواعد أكثر تعقيداً مثل:

- تاريخ الإنتاج يجب أن يكون بعد سنة معينة.
- قيمة الحقل يجب أن تنتمي إلى مجموعة محددة من القيم.
- وجود علاقة بين حقل وآخر داخل الكيان نفسه.

فوائد النموذج الغني:

عندما تصبح هذه المعلومات جزءاً من النموذج، يستطيع محرك MDA الاستفادة منها لتوليد عناصر متعددة بشكل آلي، مثل:

- واجهات المستخدم المناسبة.
- عناصر الإدخال مثل CheckBox و ComboBox.
- قيود قواعد البيانات (Database Constraints).
- عمليات التحقق من صحة البيانات.
- الاستثناءات والتنبيهات البرمجية.

وبذلك يصبح النموذج هو المصدر الأساسي للحقيقة داخل النظام، بينما تتحول بقية المكونات إلى تفاصيل تنفيذية يتم توليد جزء كبير منها تلقائياً.

كما أن قراءة نموذج غني بالمعلومات تكون أسهل بكثير من قراءة توابع طويلة مليئة بالشروط المتداخلة، الأمر الذي يسهل فهم النظام وصيانه ويقلل احتمال حدوث الأخطاء أثناء التطوير.

المجموعات القابلة للعد بشكل عودي (Recursively Enumerable Sets)

تعتمد البرمجة في جوهرها على إيجاد علاقة بين الدخل (Input) والخرج (Output)، أي تحديد النتائج التي يجب أن ينتجها النظام عند تحقق شروط أو معطيات معينة. ولكي نستطيع بناء البرامج بصورة صحيحة، لا بد أولاً من فهم طبيعة مجموعات الدخل التي نتعامل معها.

و تُصنف مسائل البرمجة إلى ثلاثة أنواع رئيسية:

1. مجموعات قابلة للعد (Countable Sets) - برامج بسيطة جداً-
2. مجموعات قابلة للعد بشكل عودي (Recursively Enumerable Sets) - تمثل 99% من البرمجيات-
3. مجموعات غير قابلة للعد (Non-Enumerable Sets) - مسائل ذكاء صناعي-

وبما أن معظم البرمجيات العملية تنتمي إلى **الفئة الثانية**، أي المجموعات القابلة للعد بشكل عودي، فإننا نحتاج إلى أسلوب يسمح بتمثيل مجموعات لا نهائية أو ضخمة باستخدام عدد محدود من القواعد. وهنا تظهر أهمية الاستقراء (Induction) الذي يوفر طريقة منهجية لتعريف هذه المجموعات والتعامل معها برمجياً دون الحاجة إلى تعداد جميع عناصرها.

النجاح ليس ضربة حظ، بل تراكم محاولات صغيرة

الاستقراء البنيوي والتقطيع البنيوي

بعد أن تعرفنا على مفهوم المجموعات القابلة للعد بشكل عودي (Recursively Enumerable Sets) ودورها في بناء معظم الأنظمة البرمجية، ننتقل إلى الآلية التي تسمح بتمثيل هذه المجموعات والتعامل معها بصورة منهجية، وهي الاستقراء البنيوي (Structural Induction) و التقطيع البنيوي (Structural Decomposition).

- تكمُن أهمية هذين المفهومين في أن معظم البنى المستخدمة في البرمجة، مثل الأعداد الطبيعية والقوائم والأشجار، يمكن تعريفها والتعامل معها باستخدام عدد قليل من القواعد البسيطة بدلاً من محاولة تعداد جميع الحالات الممكنة.

أولاً: الاستقراء البنيوي (Structural Induction):

يعتمد الاستقراء البنيوي على تعريف المجموعات باستخدام عنصر أساسي يسمى البديهية (Axiom) ومجموعة من قواعد البناء (Construction Rules).

وبهذه الطريقة يمكن وصف عدد غير محدود من العناصر باستخدام عدد محدود من القواعد.

مثال 1: الأعداد الطبيعية N

يمكن تعريف مجموعة الأعداد الطبيعية بالشكل التالي:

- البديهية:

الصفر عدد طبيعي: $Axiom: 0 \in N$

- قاعدة البناء:

إذا كان العدد x ينتمي إلى مجموعة الأعداد الطبيعية، فإن العدد $x+1$ ينتمي إليها أيضاً: $Rule1: x \in N \Rightarrow x+1 \in N$

- انطلاقاً من هاتين القاعدتين فقط يمكن توليد جميع الأعداد الطبيعية: 0, 1, 2, 3, ... دون الحاجة إلى تعريف كل عدد على حدة.

مثال 2: القوائم L

يمكن تعريف مجموعة القوائم بالطريقة نفسها:

- البديهية:

القائمة الفارغة تنتمي إلى مجموعة القوائم: $Axiom: [] \in L$

- قاعدة البناء:

إذا كانت L قائمة صحيحة، وكان x عنصراً ما، فإن إضافة x إلى بداية القائمة ينتج قائمة صحيحة أيضاً:

$Rule1: l \in L \& x \in N \Rightarrow [x | l] \in L$

- وبذلك يمكن بناء أي قائمة مهما كان طولها اعتماداً على هاتين القاعدتين فقط.

المجموعات المعجمة GL

يمكن توسيع الفكرة لتشمل بنى أكثر تعقيداً، مثل القوائم التي تحتوي داخلها على قوائم أخرى، فالقائمة قد تحتوي:

- أعداداً.
- قوائم أخرى.
- قوائم داخل قوائم بشكل متكرر.

$$Axiom: [] \in GL$$

$$Rule 1: L \in GL \ \& \ x \in N \Rightarrow [x \mid L] \in GL$$

$$Rule 2: L \in GL \ \& \ c \in GL \Rightarrow [c \mid L] \in GL$$

ويشبه هذا تماماً طريقة عمل JSON المستخدمة في الأنظمة الحديثة. ففي JSON يمكن أن تكون قيمة الحقل:

- قيمة بسيطة (رقم أو نص).
- مصفوفة (Array).
- كائناً (Object).
- أو بنية تحتوي داخلها كائنات ومصفوفات أخرى.

على سبيل المثال:

```
{
  "name": "Ahmad",
  "courses": [
    {
      "name": "Database",
      "grades": [80, 90, 95]
    },
    {
      "name": "Programming",
      "grades": [85, 88]
    }
  ]
}
```

- في هذا المثال يحتوي الكائن على قائمة، وتحتوي القائمة على كائنات أخرى، وهذه الكائنات تحتوي بدورها على قوائم جديدة. ورغم تعقيد البنية، إلا أنها كلها مبنية من عدد محدود جداً من القواعد الأساسية.
- ولهذا السبب JSON تمثل نموذجاً عاماً يمكنه وصف معظم هياكل البيانات المستخدمة في التطبيقات الحديثة، سواء كانت سجلات طلاب أو بيانات شركات أو مستندات أو حتى رسائل متبادلة عبر الشبكات. أي أن عدداً قليلاً من قواعد البناء قادر على تمثيل عدد هائل من التراكمات المختلفة، وهو نفس المبدأ الذي يقوم عليه الاستقرار البنوي.

ثانياً: التقطيع البنيوي (Structural Decomposition)

إذا كان الاستقراء البنيوي يوضح كيفية بناء الكائنات، فإن التقطيع البنيوي يمثل العملية المعاكسة، أي كيفية تفكيك الكائن إلى مكوناته الأساسية.

فبدلاً من التفكير بكيفية إنشاء الغرض، نفكر بكيفية التعبير عنه بدلالة أجزاء أبسط منه.

مثال 1: الأعداد

يمكن التعبير عن أي عدد طبيعي بالشكل:

■ العدد صفر.

■ أو العدد الحالي = العدد السابق + 1 $X = (X - 1) + 1$

أي أن العدد يمكن دائماً إرجاعه إلى حالة أبسط.

مثال 2: القوائم

يمكن تفكيك أي قائمة إلى جزأين:

3. العنصر الأول (First).

4. بقية القائمة (Rest).

فعلى سبيل المثال: [5, 8, 3, 2]

يمكن كتابتها على شكل:

■ First = 5

■ Rest = [8, 3, 2]

أو

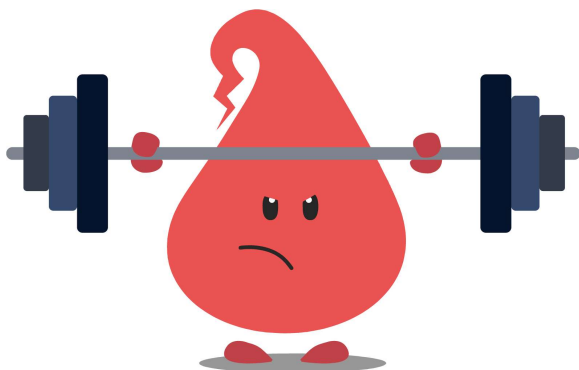
■ [5 | (8 3 2)]

ثم يمكن تطبيق العملية نفسها على بقية القائمة بشكل متكرر.

تكمُن أهمية هذا الأسلوب في أنه يسمح بحل مسائل معقدة باستخدام عدد قليل جداً من القواعد العامة بدلاً من كتابة حلول مستقلة لكل حالة ممكنة.

السقوط جزء من التعلم،

أما الاستسلام فهو القرار الحقيقي بالخسارة.



من الاستقرار إلى البرمجة

بعد تعريف البنى باستخدام الاستقرار البنيوي وتفكيكها باستخدام التقطيع البنيوي، يمكن تحويل هذه الأفكار مباشرة إلى خوارزميات وبرامج عودية (Recursive Programs).

■ القاعدة الأساسية هي:
إذا أمكن كتابة الغرض بدلالة مكونات أصغر منه، فإن حل المشكلة على الغرض الكامل يمكن بناؤه من حلول المشكلة على هذه المكونات الأصغر.

$$\text{if } X = X_0 \circ X_1 \circ X_2 \circ \dots \circ X_n \Rightarrow f(X) = g(f(X_0), f(X_1), f(X_2), \dots, f(X_n))$$

المعنى المبسط:

إذا كان لدينا عنصر مركب xxx مكوّن من أجزاء أصغر:

$$\text{if } X = X_0 \circ X_1 \circ X_2 \circ \dots \circ X_n$$

(الرمز $\circ$ يعني عملية تركيب أو دمج، حسب السياق)

وكان لدينا دالة f نريد تطبيقها على العنصر الكامل X ، فبدل أن نطبّقها مباشرة:

1. نطبّق f على كل جزء منفصل:

$$f(X_0), f(X_1), f(X_2), \dots, f(X_n)$$

2. ثم نجمع النتائج بدالة أخرى g :

$$f(X) = g(f(X_0), f(X_1), f(X_2), \dots, f(X_n))$$

أي أن التابع يعمل على الحالة الحالية، ثم يستدعي نفسه لمعالجة الأجزاء الأبسط، وبعد ذلك يقوم بدمج النتائج.

ويسمى الجزء المسؤول عن دمج النتائج أحياناً تابع الدمج (Combining Function).

مثال 1: حساب العامل (Factorial)

لحساب: $n!$

$$n! = n \times (n-1)!$$

أي أننا نحول المشكلة إلى نسخة أصغر منها.

ثم نتابع العملية حتى نصل إلى الحالة الأساسية: $0! = 1$

وفي هذه الحالة تكون عملية الضرب هي آلية دمج النتائج.

مثال 2: مجموع عناصر قائمة

لحساب مجموع عناصر قائمة:

- نأخذ العنصر الأول.
- نحسب مجموع بقية العناصر عودياً.
- نجمع النتيجة.

أي:

مجموع القائمة = العنصر الأول + مجموع بقية القائمة

وتكون عملية الجمع هنا هي آلية الدمج.

مثال 3: البحث داخل قائمة

للبحث عن عنصر معين:

- نفحص العنصر الأول.
- إذا وجدناه تنتهي العملية.
- وإذا لم نجده نعيد تنفيذ عملية البحث على بقية القائمة.

وبذلك تتحول عملية البحث بالكامل إلى سلسلة من الاستدعاءات العودية المبنية على التقطيع البنيوي للقائمة.

فوائد الاستقراء في كتابة البرامج

لا تقتصر أهمية الاستقراء البنيوي على كونه وسيلة رياضية لتعريف المجموعات، بل يمثل أيضاً منهجية موحدة لكتابة البرامج. فعندما تُبنى الخوارزميات اعتماداً على البنية الاستقرائية للبيانات، يصبح حل المشكلات أكثر تنظيماً وأقل اعتماداً على التفكير الخاص بكل حالة على حدة. ويحقق هذا الأسلوب عدداً من الفوائد المهمة، منها:

1. توحيد طريقة كتابة البرامج:

إذ يمكن تطبيق المبدأ نفسه على الأعداد والقوائم والأشجار وغيرها من البنى، مما يوفر أسلوباً عاماً لحل المشكلات.

2. زيادة سرعة التطوير:

لأن المبرمج لا يحتاج إلى ابتكار حل جديد لكل مشكلة، بل يعتمد على قواعد ثابتة قائمة على الحالة الأساسية والحالة العودية.

3. تسهيل الصيانة والتطوير:

فالكوود الناتج يكون أكثر تنظيماً وأسهل فهماً وتعديلاً من الحلول التي تعتمد على عدد كبير من الشروط الخاصة.

4. تحسين العمل الجماعي:

عندما يعتمد جميع أعضاء الفريق منهجية موحدة في تصميم الخوارزميات، يصبح فهم الكود وتعديله من قبل الآخرين أكثر سهولة.

5. تسهيل نقل البرمجيات بين الأجيال والمنصات المختلفة:

لأن الحل يعتمد على مبادئ رياضية ومنطقية عامة غير مرتبطة بلغة برمجة محددة.

ورغم أن نظرية الاستقراء البنيوي مستقلة عن لغات البرمجة، فإن تطبيقها يظهر بشكل واضح في بعض اللغات التي بُنيت أساساً على مفاهيم المنطق والذكاء الاصطناعي، مثل لغة Prolog ولغة Lisp، حيث تشكل البنى العودية والاستدلال المنطقي جزءاً أساسياً من طريقة البرمجة فيها.

ملاحظة:

تتضمن سلايدات المادة (من 77 إلى 85) أمثلة تطبيقية على هذه المفاهيم باستخدام كل من Prolog و Java، بهدف توضيح كيفية تحويل المبادئ النظرية للاستقراء البنيوي إلى برامج فعلية في لغات برمجة مختلفة.

